

EOD Report – Jenkins Tasks

Date: 1 october 2026

Environment: AWS EC2 / Jenkins 2.584

1. Configure 2 Slave Machines in the Jenkins Master

Steps on Slave Machines

- Logged in to both slave/agent EC2 machines.
- Installed Java/JDK required for Jenkins agents.
- Created the .ssh directory and configured SSH access.
- Generated SSH keys using ssh-keygen.
- Added the public key to authorized_keys.
- Configured proper permissions for .ssh and authorized_keys.
- Created the Jenkins agent working directory.

Example:

```
mkdir -p ~/.ssh  
chmod 700 ~/.ssh  
ssh-keygen -t rsa -b 4096  
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys  
chmod 600 ~/.ssh/authorized_keys
```

Steps on Jenkins Master

- Created /var/lib/jenkins/.ssh.
- Added the slave host keys to known_hosts.
- Changed ownership to the jenkins user.
- Configured SSH credentials in Jenkins.

Example:

```
mkdir -p /var/lib/jenkins/.ssh  
ssh-keyscan -H SLAVE-IP >> /var/lib/jenkins/.ssh/known_hosts  
chown jenkins:jenkins /var/lib/jenkins/.ssh/known_hosts  
chmod 600 /var/lib/jenkins/.ssh/known_hosts
```

Jenkins GUI

- Created two permanent agents under **Manage Jenkins → Nodes**.
- Configured the slave public IP/hostname.
- Selected **Launch agents via SSH**.
- Added SSH username and private-key credentials.
- Selected **Known hosts file Verification Strategy**.
- Configured the remote root directory.
- Verified that both slave machines came **Online**.

The image shows two screenshots from a Windows desktop environment.

The top screenshot is of the AWS Amazon MQ console. The browser address bar shows `https://us-east-1.console.aws.amazon.com/amazon-mq/home?region=us-east-1#/brokers`. The page title is "Amazon MQ" and the sub-header is "Brokers". A green notification banner at the top states: "Broker mybroker01 is being created. Broker creation takes about 20 minutes. Your broker's configuration can be found on the Configurations page." Below this, the section "Brokers for us-east-1 (1)" contains a table with one broker:

Name	Status	Creation time (Local)	Broker engine	Deployment mode
mybroker01	Running	September 30, 2026 at 14:44 (UTC+5:30)	RabbitMQ	Single-instance broker

The bottom screenshot is of the Jenkins web interface for a job named "job-slave1". The browser address bar shows `http://3.235.4.142:8080/job/job-slave1/`. The page shows the job's status as "Running" with a green checkmark. Under the "Permalinks" section, there are four links: "Last build (#1), 1 hr 30 min ago", "Last stable build (#1), 1 hr 30 min ago", "Last successful build (#1), 1 hr 30 min ago", and "Last completed build (#1), 1 hr 30 min ago". A "Builds" section shows a list of builds with a filter input and a dropdown menu set to "Today". The first build is "#1 10:41 AM" with a green checkmark.

2. Configure Webhooks to the Jenkins Job

- Configured the Jenkins job for webhook-based triggering.
- Enabled the required webhook trigger under **Build Triggers**.
- Added the Jenkins webhook endpoint in the Git repository.
- Configured the required repository event, such as **Push**.
- Made a change to the repository and verified that Jenkins automatically triggered the job.
- Verified the webhook delivery and Jenkins build.

Steps

Configure Webhooks to Jenkins Job

1. Install Git

On Jenkins/agent server:

```
sudo dnf install git -y
```

Check:

```
git --version
```

2. Open Jenkins Job

Go to:

Jenkins → job-slave1 → Configure

3. Select Git

Under **Source Code Management**:

- Select **Git**
- Add GitHub Repository URL
- Select credentials if the repo is private
- Branch: */master

4. Enable Webhook

Under **Triggers**, select:

☒ **GitHub hook trigger for GITScm polling**

Click **Save**.

5. Add Webhook in GitHub

Go to:

GitHub → Repository → Settings → Webhooks → Add webhook

Payload URL:

<http://3.235.4.142:8080/github-webhook/>

Content type:

application/json

Select:

☒ **Just the push event**

☒ **Active**

Click **Add webhook**.

6. Test

Make a change and push:

```
git add .
```

```
git commit -m "webhook test"
```

```
git push
```

Jenkins should automatically start the job.

Simple Flow

Git Push

↓

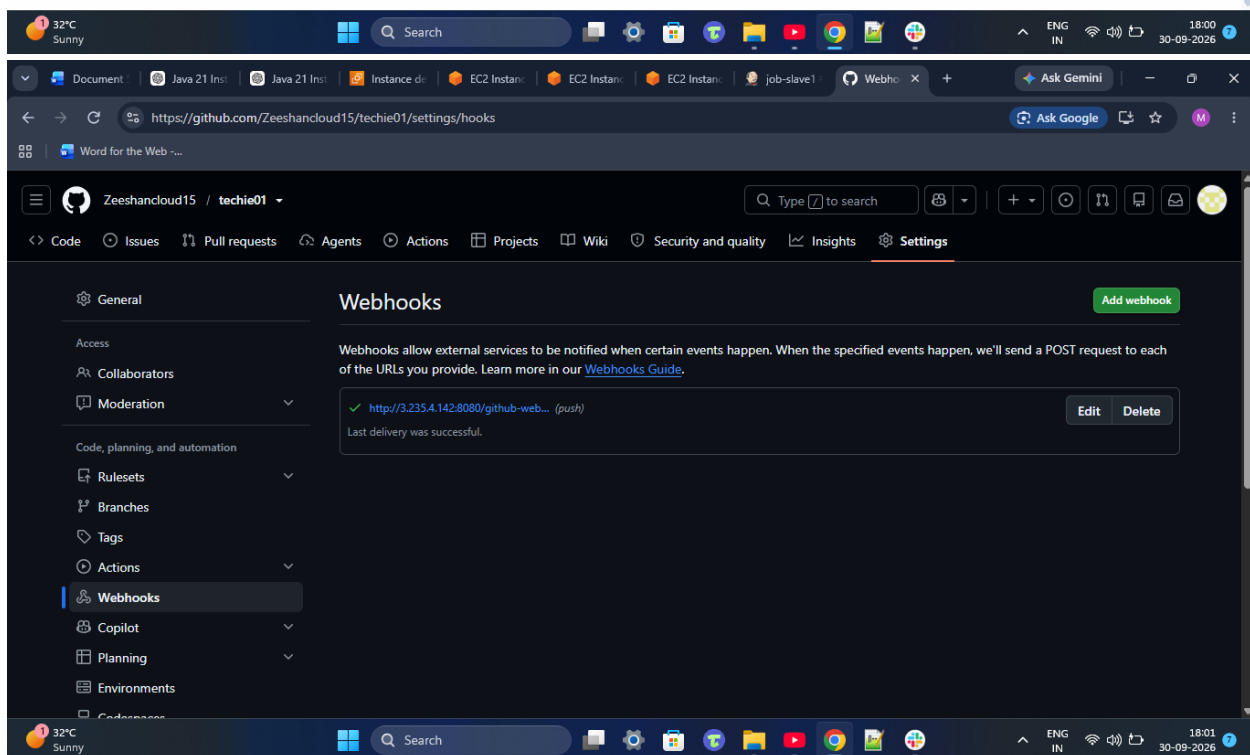
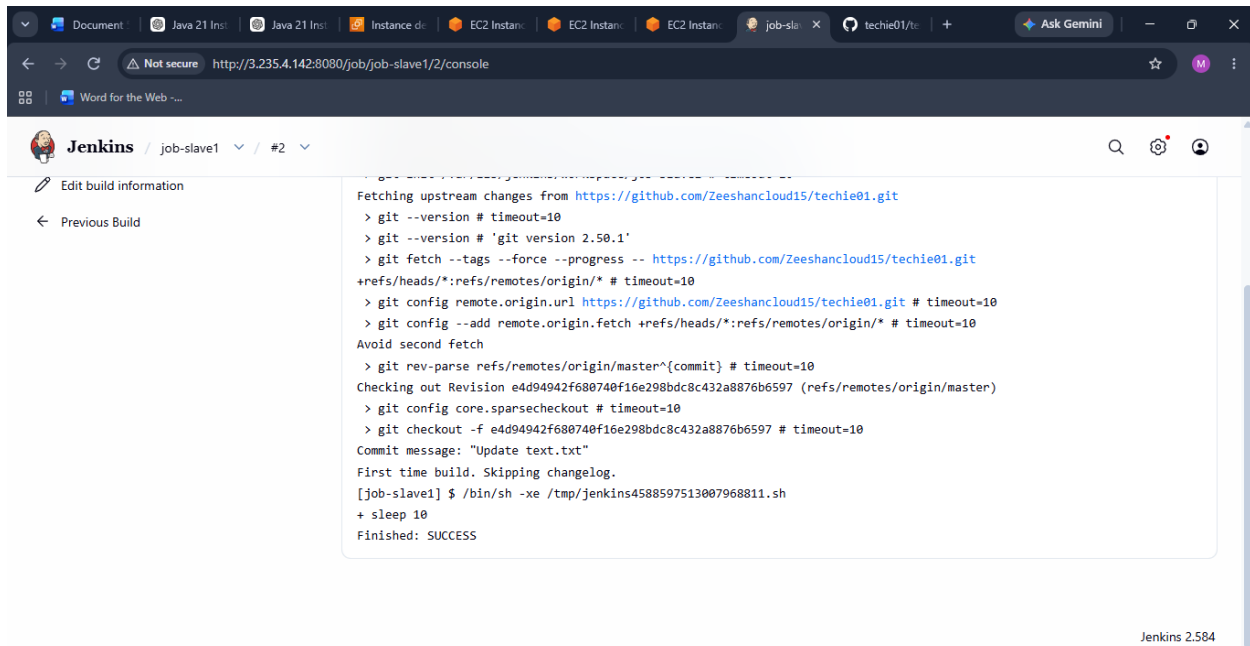
GitHub

↓

Webhook

↓
Jenkins

↓
Job runs automatically



3. Configure Poll SCM and Build Periodically Options

Poll SCM

- Opened **Jenkins Job** → **Configure** → **Build Triggers**.
- Enabled **Poll SCM**.
- Configured a schedule to periodically check the Git repository.

Example:

The top screenshot shows the Jenkins 'Configure' page for a job named 'job-slave1'. The 'Triggers' tab is selected in the left sidebar. The 'Poll SCM' checkbox is checked, and the 'Schedule' field contains '*****'. A warning message is displayed below the field: 'Do you really mean "every minute" when you say "*****"? Perhaps you meant "H*****" to poll once per hour. Would last have run at Wednesday, September 30, 2026, 12:35:00 PM Coordinated Universal Time; would next run at Wednesday, September 30, 2026, 12:36:00 PM Coordinated Universal Time.' The bottom screenshot shows the 'Console' view for the same job, displaying the output of a Git build process. The output includes commands like 'git rev-parse', 'git config', 'git fetch', 'git checkout', and 'git rev-list', along with their respective outputs and a final 'Finished: SUCCESS' message.

Build Periodically

- Enabled **Build Periodically**.
- Configured a scheduled build.

Example:

Jenkins / job-slave1 / Configure

Configure

- General
- Source Code Management
- Triggers**
- Environment
- Build Steps
- Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☒ Build periodically ?

Schedule ?

⚠ Do you really mean "every minute" when you say "*****"? Perhaps you meant "H **** *" to poll once per hour
Would last have run at Wednesday, September 30, 2026, 12:41:00 PM Coordinated Universal Time; would next run at Wednesday, September 30, 2026, 12:42:00 PM Coordinated Universal Time.

- ☐ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Save **Apply**

Jenkins

- + New Item
- Build History
- Project Relationship
- Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

Build-In Node	Progress	Count
job-slave1		#4
slave01		0/2
slave02		0/1

Icon: S M L

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	job-slave1	3 min 2 sec #3	N/A	10 sec

REST API Jenkins 2.584

4. Take a Backup of Jenkins Server Using Bash Script

- Created a backup directory.
- Created a Bash script to back up the Jenkins home directory.
- Used tar to create a compressed backup.
- Added the date and time to the backup filename.
- Executed the script and verified the backup archive.

Jenkins home directory:

/var/lib/jenkins

```
#!/bin/bash
```

```
BACKUP_DIR="/backup"
```

JENKINS_HOME="/var/lib/jenkins"

DATE=\$(date +%F-%H%M%S)

```
tar -czf "$BACKUP_DIR/jenkins-backup-$DATE.tar.gz" "$JENKINS_HOME"
```

```
echo "Jenkins backup completed successfully."
```

Made the script executable:

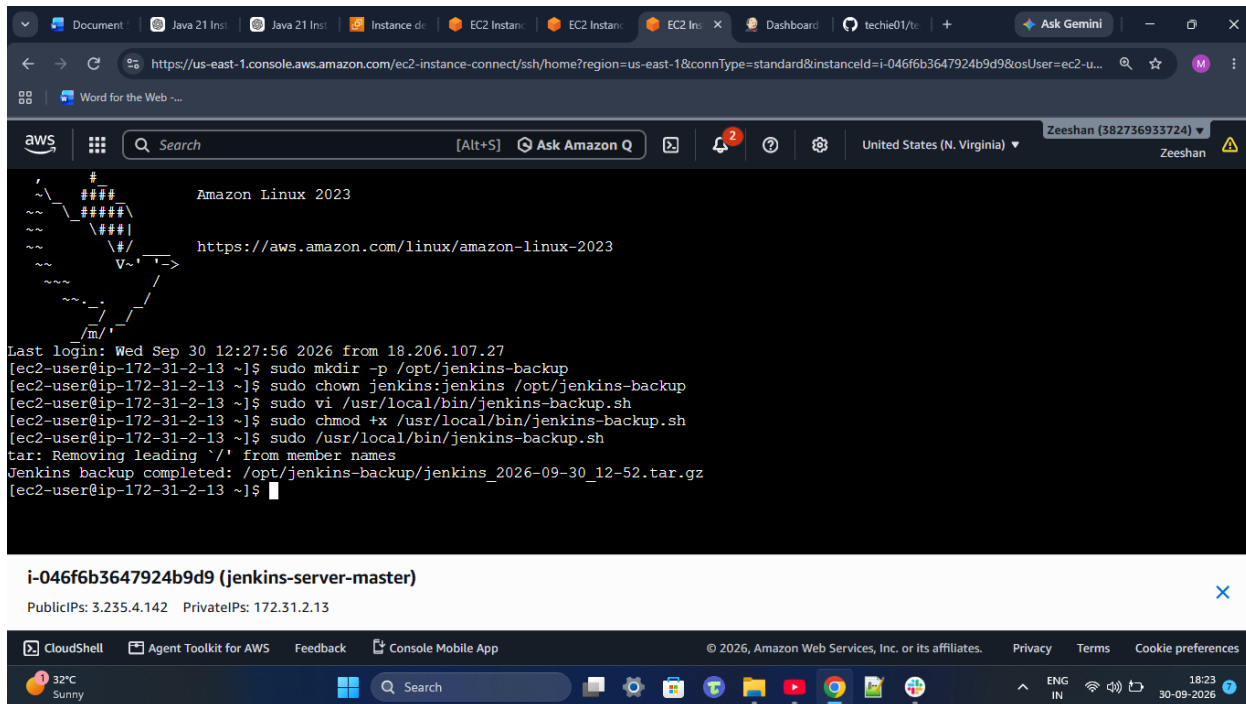
```
chmod +x /backup/jenkins-backup.sh
```

Executed:

```
sudo /backup/jenkins-backup.sh
```

Verified:

```
ls -lh /backup/
```



```
/usr/local/bin/
```

└─jenkins-backup.sh

```
/opt/jenkins-backup/
```

```
└─jenkins_2026-09-30_02-00.tar.gz
```

```
└─jenkins 2026-10-01 02-00.tar.gz
```

jenkins_2026-10-02_02-00.tar.gz

/var/lib/jenkins/
└─ Jenkins Home

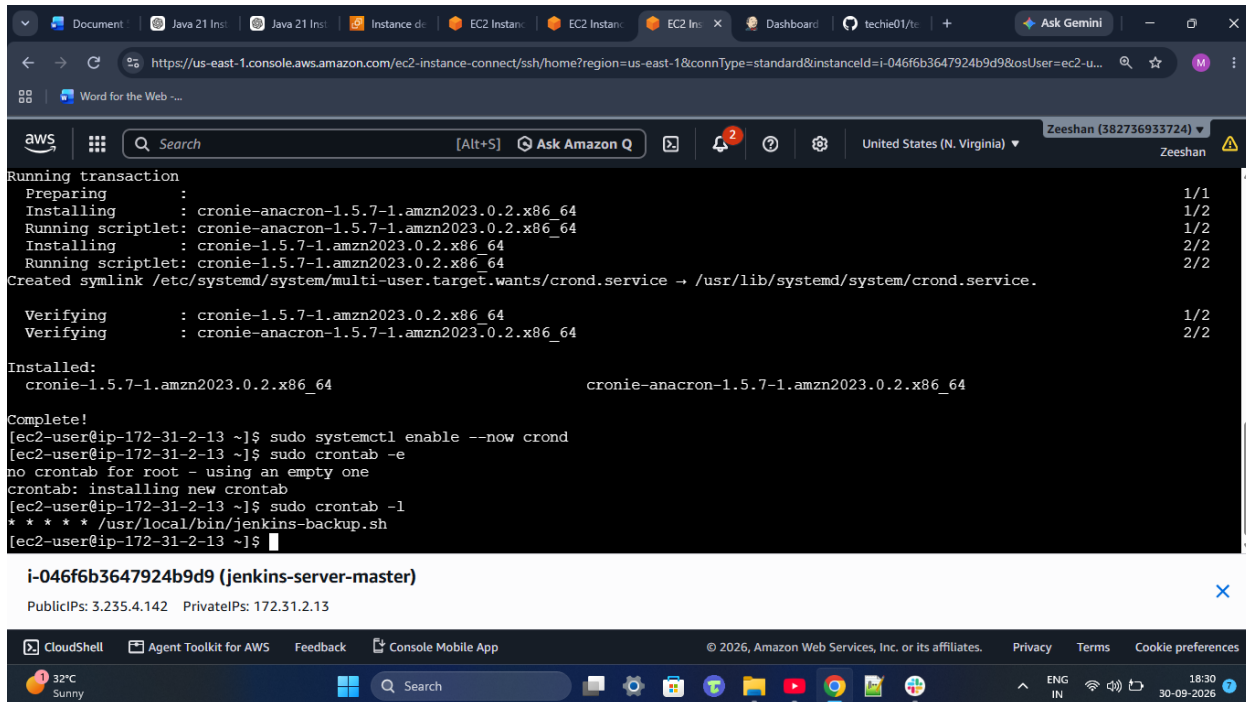
For **cronjob** installation

sudo dnf install cronie -y ----- AWS EC2 LINUX

sudo systemctl enable --now crond

sudo crontab -e ----- 0 2 * * * /usr/local/bin/jenkins-backup.sh

sudo crontab -l -----verify



The screenshot shows an AWS EC2 console terminal window. The terminal output displays the installation of cronie and anacron on an Amazon Linux 2 instance. The steps include running 'dnf install cronie -y', enabling the crond service with 'systemctl enable --now crond', and editing the crontab with 'crontab -e'. The crontab is configured with a cron job to run '/usr/local/bin/jenkins-backup.sh' at 02:00 every day. The terminal also shows the verification of the crontab with 'crontab -l'.

```
Running transaction
Preparing                : cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64      1/1
Installing               : cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64      1/2
Running scriptlet        : cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64      1/2
Installing               : cronie-1.5.7-1.amzn2023.0.2.x86_64             2/2
Running scriptlet        : cronie-1.5.7-1.amzn2023.0.2.x86_64             2/2
Created symlink /etc/systemd/system/multi-user.target.wants/crond.service → /usr/lib/systemd/system/crond.service.

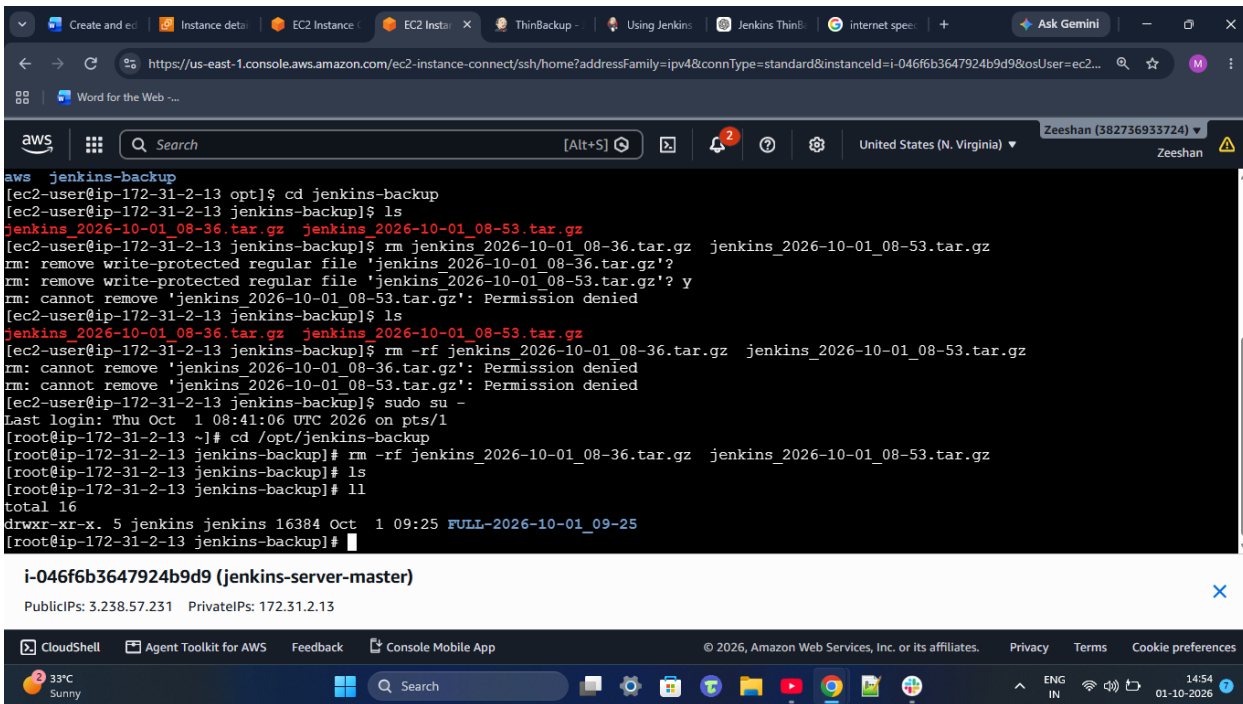
Verifying                : cronie-1.5.7-1.amzn2023.0.2.x86_64             1/2
Verifying                : cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64      2/2

Installed:
cronie-1.5.7-1.amzn2023.0.2.x86_64      cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64

Complete!
[ec2-user@ip-172-31-2-13 ~]$ sudo systemctl enable --now crond
[ec2-user@ip-172-31-2-13 ~]$ sudo crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
[ec2-user@ip-172-31-2-13 ~]$ sudo crontab -l
* * * * * /usr/local/bin/jenkins-backup.sh
[ec2-user@ip-172-31-2-13 ~]$
```

5. Take a Backup of Jenkins Using ThinBackup Plugin

- Installed the **ThinBackup** plugin from Jenkins Plugin Manager.
- Opened the ThinBackup configuration.
- Configured the backup directory.
- Configured the backup schedule.
- Performed a manual backup.
- Verified that the backup files were created successfully.
- Reviewed the available restore option.



6. Set Up a New Jenkins Server and Restore the Backup Taken in Task 4

Objective

Set up a new Jenkins server and restore the Jenkins backup taken in Task 4 using two methods:

1. Transfer the backup using **SCP**
2. Restore the backup using the **ThinBackup plugin**

Method 1: Transfer Backup Using SCP

Step 1: Verify the Backup

On the old Jenkins server:

```
cd /opt/jenkins-backup
```

```
ls -lh
```

Example:

```
jenkins_2026-10-01_09-37.tar.gz
```

Step 2: Copy Backup to New Jenkins Server

Use SCP:

```
scp -i backup.pem jenkins_2026-10-01_09-37.tar.gz ec2-user@34.230.10.90:/home/ec2-user/
```

Where:

- scp → securely copies files between servers
- -i backup.pem → SSH private key
- jenkins_2026-10-01_09-37.tar.gz → Jenkins backup
- [ec2-user@34.230.10.90](#) → new Jenkins server
- /home/ec2-user/ → destination directory

Step 3: Verify on New Server

SSH into the new server:

```
ssh -i backup.pem ec2-user@34.230.10.90
```

Check:

```
ls -lh /home/ec2-user/
```

The backup should be present:

```
jenkins_2026-10-01_09-37.tar.gz
```

Method 2: Restore Using ThinBackup Plugin

Step 1: Install Jenkins

Java and Jenkins were already installed on the new server.

Verify Jenkins:

```
sudo systemctl status jenkins
```

Step 2: Install ThinBackup Plugin

In the new Jenkins server:

Manage Jenkins → Plugins → Available plugins

Search:

ThinBackup

Install the plugin.

Step 3: Put Backup in ThinBackup Directory

Create the ThinBackup directory:

```
sudo mkdir -p /opt/jenkins-backup
```

Give Jenkins ownership:

```
sudo chown -R jenkins:jenkins /opt/jenkins-backup
```

Copy the transferred backup:

```
sudo cp /home/ec2-user/jenkins_2026-10-01_09-37.tar.gz /opt/jenkins-backup/
```

Verify:

```
sudo ls -lh /opt/jenkins-backup/
```

Step 4: Configure ThinBackup

Go to:

Manage Jenkins → System

Find the **ThinBackup** section.

Set:

Backup directory:

```
/opt/jenkins-backup
```

Click **Save**.

Step 5: Open ThinBackup Restore

Go to:

Manage Jenkins → ThinBackup

Click:

Restore

The transferred backup should now be available in the restore list.

Select:

jenkins_2026-10-01_09-37.tar.gz

Select the appropriate restore option and start the restore.

Step 6: Restart Jenkins

After restoration:

```
sudo systemctl restart jenkins
```

Check:

```
sudo systemctl status jenkins
```

Expected:

Active: active (running)

Step 7: Verify Restored Jenkins

Login to the new Jenkins server and verify:

- Jobs
- Users
- Agents/nodes
- Jenkins configuration
- Plugin configuration
- Job configuration

Document: Instance EC2 I... EC2 Inst... EC2 Inst... Nodes - Jenkins T internet Linux Sign in - Ask Gemini

https://us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-00628c149edee9e1f&osUser=ec2-us...

Word for the Web ...

aws Search [Alt+S] Ask Amazon Q United States (N. Virginia) Zeeshan (382736933724) Zeeshan

```
drwxr-xr-x. 4 jenkins jenkins 36 Sep 30 11:14 nodes
-rw-r--r--. 1 jenkins jenkins 170 Oct 1 09:24 org.jenkinsci.plugins.displayurlapi.DefaultDisplayURLProviderGlobalConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 289 Oct 1 09:24 org.jenkinsci.plugins.github_branch_source.GitHubConfiguration.xml
-rw-r--r--. 1 jenkins jenkins 169 Oct 1 09:24 org.jenkinsci.plugins.workflow.flow.GlobalDefaultFlowDurabilityLevel.xml
-rw-r--r--. 1 jenkins jenkins 229 Oct 1 09:24 org.jenkinsci.plugins.workflow.libs.GlobalLibraries.xml
-rw-r--r--. 1 jenkins jenkins 247 Oct 1 09:24 org.jenkinsci.plugins.workflow.libs.GlobalUntrustedLibraries.xml
-rw-r--r--. 1 jenkins jenkins 1089 Oct 1 09:24 org.jvnet.hudson.plugins.thinbackup.ThinBackupPluginImpl.xml
drwxr-xr-x. 96 jenkins jenkins 16384 Oct 1 08:04 plugins
-rw-r--r--. 1 jenkins jenkins 259 Oct 1 08:10 queue.xml
-rw-r--r--. 1 jenkins jenkins 259 Oct 1 08:07 queue.xml.bak
-rw-r--r--. 1 jenkins jenkins 64 Sep 30 06:13 secret.key
-rw-r--r--. 1 jenkins jenkins 0 Sep 30 06:13 secret.key.not-so-secret
drwxr-xr-x. 2 jenkins jenkins 16384 Sep 30 10:41 secrets
drwxr-xr-x. 2 jenkins jenkins 149 Oct 1 08:02 updates
drwxr-xr-x. 2 jenkins jenkins 24 Sep 30 06:13 userContent
drwxr-xr-x. 5 jenkins jenkins 16384 Sep 30 12:38 users
drwxr-xr-x. 3 jenkins jenkins 24 Sep 30 12:30 workspace
[root@ip-172-31-23-113 jenkins]# cd workspace
[root@ip-172-31-23-113 workspace]# ll
total 0
drwxr-xr-x. 6 jenkins jenkins 152 Sep 30 12:38 job-slave1
[root@ip-172-31-23-113 workspace]#
```

i-00628c149edee9e1f (slave-02)

PublicIPs: 34.230.10.90 PrivateIPs: 172.31.23.113

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

33°C Sunny

Search

Windows Taskbar

Document: Instance EC2 I... EC2 Inst... EC2 Inst... Nodes - Jenkins T internet Linux Sign x - Ask Gemini

Not secure http://34.230.10.90:8080/login?from=%2F

Word for the Web ...



Sign in to Jenkins

Username

Password

☐ Keep me signed in

Sign in

Documents

Instance

EC2 Inst

EC2 Inst

EC2 Inst

Nodes

Jenkins

internet

Linux

Dash

Ask Gemini

Not securehttp://34.230.10.90:8080

Word for the Web

Jenkins

+ New Item

Build History

Project Relationship

Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

Built-In Node0/2

slave010/2

slave02(launching...)

All

+

S	W	Name	Last Success	Last Failure	Last Duration	
...	☀	backup-job	N/A	N/A	N/A	▶
✓	☀	job-slave1	1 hr 58 min #34	N/A	10 sec	▶

Icon: S M L

REST API

Jenkins 2.584

33°C Sunny

Search

Settings

Calendar

File Explorer

YouTube

Chrome

Edge

Firefox

VS Code

ENG IN

15:36

01-10-2026